PAGEDATTENTION IN MINITORCH: KV CACHE EFFICIENCY, PREFIX REUSE, AND MEMORY SHARING

NingRui Yang [1] **Yixiang Dai** [1]

ABSTRACT

Large language model serving is often limited less by attention arithmetic than by the memory reserved for the key-value (KV) cache during autoregressive generation. A contiguous KV cache is easy to index, but it reserves a worst-case slab per request and wastes capacity whenever prompts and generated lengths are shorter than the cap. We implement PagedAttention in MiniTorch to expose the core systems idea behind vLLM in a compact educational codebase: store KV tensors in fixed-size physical blocks, translate logical token positions through per-sequence block tables, and share complete prefix blocks using reference counts. Our implementation includes a Python block manager, prefix-cache and branch-sharing APIs, a transformer integration, a CUDA decode kernel and runtime, a contiguous-KV baseline, and a benchmark suite. On a realistic non-aligned workload, block-granular allocation reaches 96.6% KV efficiency versus 23.4% for worst-case static reservation and 46.8% for a realistic static cap. Under a fixed KV budget, it supports up to 2.00x more concurrent short prompts, gives up to 1.90x faster cached-prefix prefill, and reduces KV blocks by up to 78–79% in parallel-sampling and beam-style branching simulations.

1 INTRODUCTION AND MOTIVATION

Autoregressive LLM inference repeatedly appends one token to each active request. To avoid recomputing all previous attention states, serving systems store each layer's key and value vectors in a KV cache. The cache grows linearly with sequence length, number of layers, and hidden size:

$$\text{bytes/token} = 2 \cdot L \cdot H \cdot d_h \cdot \text{sizeof(dtype)}.$$

At serving scale, these bytes directly determine how many requests fit on a GPU.

The simplest allocation strategy reserves a contiguous KV slab for every active request up to a maximum context or admission cap. That strategy is operationally convenient but wasteful: most requests terminate before the cap, prompt lengths are not aligned with allocator boundaries, and multi-output generation duplicates the same prompt cache. These effects reduce effective batch size even when compute units are idle.

PagedAttention (Kwon et al., 2023) changes the memory contract. Instead of requiring each request's KV cache to be physically contiguous, it partitions the cache into fixed-size blocks and uses a block table to map logical token positions to physical blocks. This project reimplements that idea inside MiniTorch, trading production-level optimization for a readable implementation that makes the allocator, cache sharing, CUDA path, and experimental tradeoffs explicit.

Our contributions are:

1. a reference-counted `BlockManager` with block tables, fragmentation accounting, prefix-cache lookup/publication, fork and clone APIs, and KV memory accounting;
2. a MiniTorch decoder integration that supports prefill, decode, prefix reuse, and generation over paged KV blocks;
3. a custom CUDA V1-style decode kernel and device-resident runtime buffers for non-contiguous KV lookup;
4. a contiguous-KV baseline and six benchmark families covering memory efficiency, capacity, decode speed, prefix prefill, parallel sampling, and beam-style branch sharing.

2 RELATED WORK AND BACKGROUND

PagedAttention and vLLM. Kwon et al. (Kwon et al., 2023) introduced PagedAttention as the central memory manager in vLLM. Their production system combines block tables, continuous batching, prefix/block sharing, scheduling, and highly optimized CUDA kernels. Our work follows

[1]ECE, Carnegie Mellon University, Pittsburgh, PA, US. **AUTHORERR: Missing \mlsyscorrespondingauthor.**

Proceedings of the 5th MLSys Conference, Santa Clara, CA, USA, 2024. Copyright 2024 by the author(s).

the same block-granular KV storage abstraction, but differs in purpose and scope: we implement it in MiniTorch with small, inspectable Python and CUDA components, and our measurements isolate the allocator effects rather than claiming production serving throughput.

Attention kernels. FlashAttention (Dao et al., 2022) and FlashAttention-2 (Dao, 2024) reduce attention IO by tiling softmax computation, while FlashInfer (Ye et al., 2025) provides modern serving-oriented attention kernels, including paged layouts. These works optimize the attention computation path. Our project instead emphasizes KV residency policy: how cache bytes are reserved, reused, shared, and reclaimed. We implement a simple V1 kernel to validate non-contiguous lookup, not to match FlashAttention-class throughput.

Serving frameworks and OS analogy. Systems such as Hugging Face Text Generation Inference (Hugging Face, 2023) and TensorRT-LLM (NVIDIA, 2024) also use KV-cache management techniques inspired by PagedAttention. The abstraction is analogous to virtual memory paging (Silberschatz et al., 2018): logical token blocks are virtual pages, physical KV blocks are page frames, and the block table is a page table. MiniTorch (Rush, 2020) gives us a small enough framework to rebuild these pieces directly.

3 METHODOLOGY

We organized the project around four research questions.

- **RQ1:** How much reserved KV memory can block-granular allocation recover under non-aligned request lengths?
- **RQ2:** Under a fixed KV-block budget, how much additional concurrency does paged allocation allow?
- **RQ3:** How much do prefix caching and branch sharing reduce repeated work and duplicated KV state?
- **RQ4:** What does a minimal CUDA implementation need in order to compute attention over non-contiguous cache blocks correctly?

3.1 Block-Granular KV Allocation

The allocator owns a global key cache and value cache for each layer, shaped `(num_blocks, block_size, n_head, head_dim)`. A request receives a `BlockTable`; token t is translated by $\lfloor t/B \rfloor$ and $t \bmod B$, where B is the block size. Allocation is demand driven: prefill allocates $\lceil T/B \rceil$ blocks for a prompt of length T, decode appends into the current tail block when possible, and retired requests decrement reference counts and return zero-reference blocks to the free list.

Internal fragmentation is the unused tail space inside allocated blocks. This creates a real block-size tradeoff: smaller blocks reduce tail waste, while larger blocks reduce table size and may improve locality. Our fragmentation and capacity experiments deliberately use non-aligned lengths so this tradeoff is visible.

3.2 Prefix Cache and Branch Sharing

The prefix cache shares only complete blocks. After prefill finishes for all layers, `publish_sequence_prefix_blocks` computes a hash chain over the full logical blocks and maps each hash to its physical block id. Later requests call `lookup_prefix_blocks`; matching blocks are inserted into the new request's block table and their reference counts are incremented. Partial tail blocks are not shared, which keeps mutation rules simple.

Branching workloads use the same ownership mechanism. `fork_sequence` models a paged branch by sharing the parent's existing block table and allocating new blocks only as children diverge. `clone_sequence` models a naive baseline that physically duplicates the prompt KV for each output. This lets us quantify memory effects for parallel sampling and beam-like generation without implementing a full scheduler.

3.3 CUDA Decode Path

The CUDA kernel assigns one thread block to each (sequence, head) pair. It performs three passes: compute $q \cdot k$ logits through block-table lookups, apply a stable softmax with warp reductions, and accumulate weighted values. The host runtime keeps cache, query, block-table, context-length, and output buffers on the device, and exposes C-callable update methods used from Python.

Our kernel is intentionally simple. It is useful for correctness and for teaching the translation from logical tokens to physical KV blocks, but it lacks vLLM-style template specialization, coalesced cache layouts, online softmax, and V2 multi-block decomposition for long contexts.

3.4 Baselines

We use two baselines. The no-cache baseline re-runs the full prompt at each decode step, which highlights the algorithmic benefit of caching but is not a fair serving implementation. The contiguous-KV baseline wraps the same `PagedDecoderLM` weights with HuggingFace-style preallocated K/V arrays shaped `(max_batch, n_head, max_seq_len, head_dim)`. It avoids recomputing the prompt but still pays static contiguous reservation.

4 Implementation Details and Code Snippets

The implementation spans `minitorch/block_manager.py`, `minitorch/transformer.py`, `minitorch/paged_attention.py`, `src/paged_attention.cu`, `project/contiguous_kv_baseline.py`, and the benchmark scripts in `project/`. Figure 1 shows the three most important mechanics: prefix-backed allocation, the CUDA block-table translation, and the runtime's device-resident cache updates.

The transformer integration separates prefill and decode. During prefill, the model checks prefix-cache hits before allocating blocks; hit groups compute only suffix tokens while attending to cached prefix K/V. During decode, each active sequence appends one logical token, writes the new K/V into its physical block slot, and calls paged attention over the full cached context.

A major engineering issue was CUDA context ownership. MiniTorch uses Numba/PyCUDA driver-API paths, PyTorch uses the CUDA runtime API, and our shared library also uses runtime calls. The fix was to compile with shared `libcudart`, initialize the runtime early, and make driver-API imports lazy. Without this, runtime calls could fail after Numba had initialized a separate context.

5 Experiments

5.1 Setup

The final benchmark suite in `project/run_rigorous_benchmark.py` runs six experiments and plots them with `project/plot_rigorous_figures.py`. Unless otherwise stated, the report-grade configuration uses `n_embd=512`, `n_head=8`, `n_layers=4`, `block_size=16`, `num_kv_blocks=512`, and one warmup plus three timed trials for timing experiments. Correctness is checked by unit tests and by benchmark rows that compare paged attention against reference outputs.

5.2 Memory Efficiency and Capacity

Figure 2 reports the central allocator result. For the realistic non-aligned workload $[48, 67, 112, 160, 256, 80, 35, 200]$, paged allocation reserves only the blocks needed for live tokens and reaches 96.6% KV efficiency. Worst-case static reservation at `n_positions=512` is only 23.4% efficient; a more realistic static cap at the maximum observed length is 46.8% efficient. The capacity curve then fixes the KV budget and asks how many concurrent sequences fit. Paged allocation supports up to 2.00x more short prompts than the static `seq_len+32` cap and remains 1.14x better at 256 tokens.

5.3 Decode and Prefix Timing

Decode timing compares paged KV against both no-cache re-prefill and contiguous KV. Paged decode is consistently faster than the no-cache baseline because it reuses K/V rather than recomputing the full prefix. Compared with the contiguous-KV baseline, the result is closer and sometimes favors the simpler contiguous path; this is expected because our educational kernel and Python runtime prioritize clarity over memory-coalesced production performance. We therefore treat Figure 3 as evidence that caching works and as a reminder that allocator wins do not automatically imply a faster kernel.

Prefix caching shows the clearest timing benefit. Requests with shared prompt prefixes reuse full cached blocks and compute only the suffix. At requested shared-prefix fractions of 0%, 25%, 50%, and 75%, the measured prefill speedups are 1.36x, 1.80x, 1.76x, and 1.90x respectively. The 0% case is not a contradiction: with block-level cache behavior and benchmark overheads, even small allocator/runtime effects can shift short prefill timings; the main trend is that more full-block sharing reduces work.

5.4 Branching Memory

Multi-output generation is a natural fit for block sharing. In parallel sampling, several completions share one prompt and diverge only after the prompt. In beam-style search, surviving hypotheses can also share a generated trunk before the final branch. Figure 4 compares naive cloning with paged fork semantics. Parallel sampling saves 33–78% of KV blocks depending on prompt length and number of outputs. The beam-style simulation saves 33–79% at peak and even more after pruning because the prompt and winning trunk are shared.

6 Analysis, Ablations, and Findings

Block size controls allocator waste. The dedicated fragmentation sweep confirms the expected tradeoff. For sequence length 33, internal fragmentation is 17.5% with block size 8 and 31.3% with block size 16. For longer or better-aligned lengths such as 100 and 130, the gap narrows. Thus, block size should be chosen with both workload length distribution and kernel/table overhead in mind.

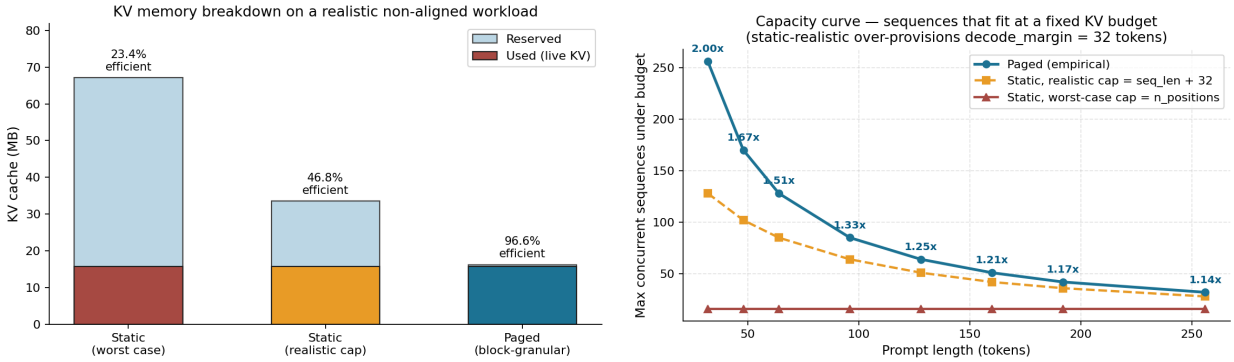
Memory gains are stronger than speed gains in this prototype. Paged allocation dramatically improves KV efficiency and fixed-budget capacity, but our decode timing is limited by Python/MiniTorch overhead and a simple CUDA path. This matches the project goal: validate the systems abstraction in a compact framework, not reproduce vLLM

```python
def allocate_sequence_with_prefix(
    seq_id, num_tokens,
    prefix_blocks):
    table = BlockTable(seq_id)
    for bid in prefix_blocks:
        table.append_block(bid)
        blocks[bid].ref_count += 1

    remaining = num_tokens - len(
    prefix_blocks) * block_size
    for _ in range(ceil(remaining /
    block_size)):
        block = allocate_block()
        table.append_block(block.
    block_id)
        block.num_filled = min(
    remaining, block_size)
        remaining -= block.
    num_filled
```

```cpp
int logical_block = token_idx /
    block_size;
int slot = token_idx % block_size;
int physical_block = seq_block_table
    [logical_block];

const float* k = key_cache
  + physical_block * (block_size *
    n_head * head_dim)
  + slot * (n_head * head_dim)
  + head_idx * head_dim;
```

```cpp
void
    paged_attention_runtime_update_slot
    (
    void* handle, int block_id, int
    slot_idx,
    const float* key_host, const
    float* value_host) {
  size_t off = ((size_t)block_id *
    block_size + slot_idx)
            * n_head * head_dim;
  cudaMemcpy(d_key_cache + off,
    key_host, slot_bytes,
            cudaMemcpyHostToDevice)
    ;
  cudaMemcpy(d_value_cache + off,
    value_host, slot_bytes,
            cudaMemcpyHostToDevice)
    ;
}
```

Figure 1. Representative implementation snippets. Left: cached prefix blocks are reused by reference and only suffix blocks are allocated. Middle: the CUDA kernel translates logical token positions into physical cache addresses. Right: the runtime updates individual block slots without reallocating the full cache.



(a) Reserved versus live KV bytes.

(b) Max concurrent sequences under a fixed KV budget.

Figure 2. Paged allocation converts worst-case per-request reservation into block-granular demand allocation.

throughput.

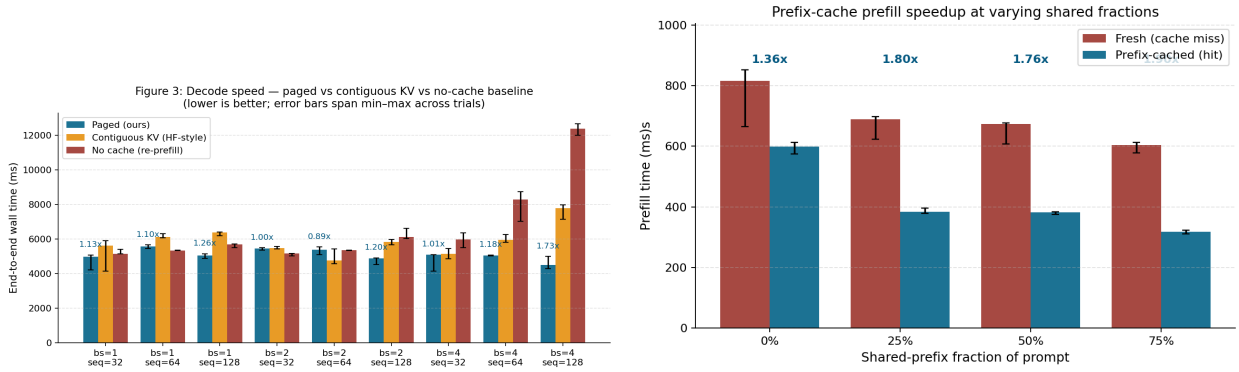
Sharing multiplies the allocator benefit. Prefix caching, parallel sampling, and beam sharing all use the same block-table/reference-count mechanism. This is the main design lesson: once requests store mappings instead of private slabs, reuse policies become metadata updates rather than tensor copies.

What did not work as production evidence. No-cache decode is useful pedagogically but overstates speedup against a real serving engine. We therefore added the contiguous-KV baseline to distinguish caching benefits from paging benefits. Similarly, the branch experiments are allocator simulations, not complete parallel sampling or beam-search schedulers.

7 Conclusion and Discussion

This project shows that PagedAttention's core idea can be rebuilt in a small teaching framework while preserving the most important systems behaviors. Block-granular KV allocation raises realistic-workload efficiency to 96.6%, doubles short-prompt capacity under a fixed KV budget, accelerates cached-prefix prefill, and makes multi-output KV sharing cheap. The main takeaway is that memory residency policy is a first-class serving optimization: it can improve admission capacity, reuse, and branching efficiency without changing model weights or attention semantics.

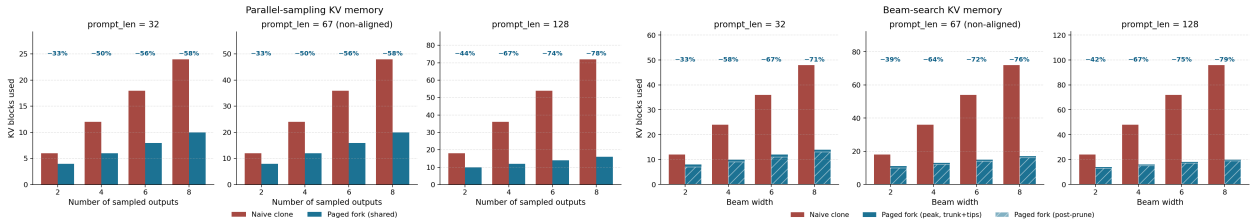
Known limitations remain. The CUDA kernel is a readable V1 implementation, not a production kernel. The cache layout is not optimized for coalescing, there is no V2 decomposition for very long contexts, and some runtime paths still perform host-device transfers that a production server would avoid. The prefix cache shares only complete blocks and lacks a full production eviction/scheduler policy. Finally,



(a) End-to-end wall time for paged, contiguous KV, and no-cache decode.

(b) Prefix-cache prefill speedup at varying shared fractions.

Figure 3. Timing experiments separate caching benefits from CUDA/kernel quality. Error bars show min–max across timed trials where available.



(a) Parallel-sampling KV blocks.

(b) Beam-style KV blocks.

Figure 4. Reference-counted block tables make prompt and trunk sharing a memory policy rather than a model change.

our report-grade branch experiments model allocator behavior, not complete request scheduling or quality-aware beam search. Future work would add GPU-resident end-to-end serving, online softmax and templated kernels, Nsight profiling, FP16/BF16 support, and a scheduler that combines paged allocation with admission control.

8 Team Member Contributions

NingRui Yang implemented and documented the allocator and model integration: the `BlockManager`, `KVBlock`, `BlockTable`, fragmentation and memory accounting, prefix-cache publication/lookup, and fork/clone sharing paths in `minitorch/block_manager.py`; the prefill/decode/generation integration in `minitorch/transformer.py`; and project documentation under `docs/` explaining the block manager, memory accounting, prefix cache, and usage workflow.

Yixiang Dai implemented and evaluated the CUDA and benchmarking paths: the V1 paged-attention kernel and runtime functions in `src/paged_attention.cu`; the Python/CUDA wrapper path used by MiniTorch; the contiguous-KV comparison baseline in `project/contiguous_kv_baseline.py`; and the benchmark/plotting scripts `project/run_benchmark.py`, `project/run_rigorous_benchmark.py`, `project/plot.py`, and `project/plot_rigorous_figures.py` that produced the final report figures.

Both members collaborated on correctness tests, debugging the CUDA runtime/driver context issue, interpreting benchmark results, building the poster, and writing this final report.

References

Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Hugging Face. Text generation inference.

https://github.com/huggingface/text-generation-inference, 2023.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*, pp. 611–626. ACM, 2023.

NVIDIA. TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM, 2024.

Rush, A. MiniTorch: A DIY teaching library for machine learning engineering. https://minitorch.github.io/, 2020.

Silberschatz, A., Galvin, P. B., and Gagne, G. *Operating System Concepts*. Wiley, 10th edition, 2018.

Ye, Z., Chen, L., Lai, R., Lin, W., Zhang, Y., Wang, S., and Chen, T. FlashInfer: Efficient and customizable attention engine for LLM inference serving, 2025.